sdhash

approximate matching

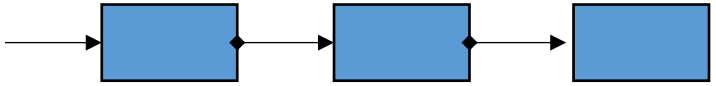
Bloom filters

Bloom filter

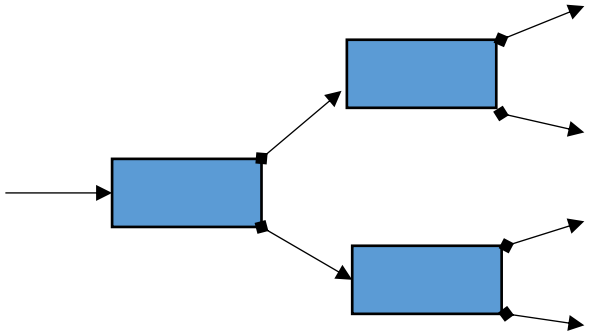
- Invented by
  - Burton Bloom [1970]
- Description
  - **SPACE-EFFICIENT PROBABILISTIC DATA STRUCTURE FOR REPRESENTING A SET**
- Rationale for existence
  - trade space efficiency for the (quantifiable) possibility of returning false positives

Representing a (non-shrinking) set

- List
  - insert: **O**(1)
  - lookup: **O**(n)

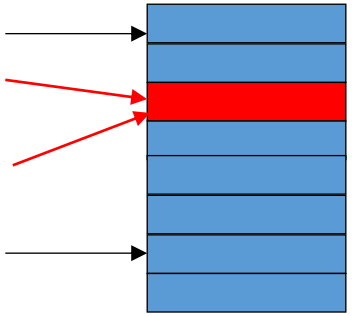


- Tree/skip list
  - insert/lookup: **O**($\log_2 n$)



- Hash table
  - insert/lookup: **O**(1)*

*collisions can be a problem



BIT HASH TABLE

- Let's allocate one bit for each possible element in the hash table: S[0 .. m-1]
  - `hash(x)`: U → [0 .. *m*-1], for all *x* in *U*; (usually, `m = 2^l`)
  - `insert(x)`:
    - return `S[hash(x)] = 1`
  - `lookup(x)`:
    - return `S[hash(x)]`

```
S: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
   hash("hello") → 3
S: 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0
   hash("world") → 15
S: 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1
   hash("collision") → 3
S: 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1
```

- Observations
  - no false negatives:
    - insert(*x*) ⇒ lookup(*x*)
    - ¬ lookup(*x*) ⇒ ¬ insert(*x*)
  - false positives are possible:
    - lookup(*x*) ⇒ insert(*x*) or { insert(*y*) and hash(*x*) == hash(*y*) }

Bloom filter construction

- Bit vector: S[0 .. m-1]
  - $h_1(x),\ldots,h_k(x)$: $U \rightarrow [0 .. m-1]$, for all x in U, $k \geq 1$
  - insert(x):

```
for i in 1..k:
    S[h_i(x)] = 1
```

  - lookup(x):

```
return S[h_1(x)] & … & S[h_k(x)]
```

- Same properties
  - no false negatives:

    insert(x) $\Rightarrow$ lookup(x)

    $\neg$ lookup(x) $\Rightarrow$ $\neg$ insert(x)
  - false positives are possible

```
S: 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

h_1("hello") → 3, h_2("hello") → 7

```
S: 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0
```

h_1("world") → 5, h_2("world") → 11

```
S: 0 0 0 1 0 1 0 1 0 0 0 1 0 0 0 0
```

(h_1,h_2)("collision") → (3,11)

```
S: 0 0 0 1 0 1 0 1 0 0 0 1 0 0 0 0
```

FALSE POSITIVE RATE

- *Pr(specific bit is zero after n insertions) =*

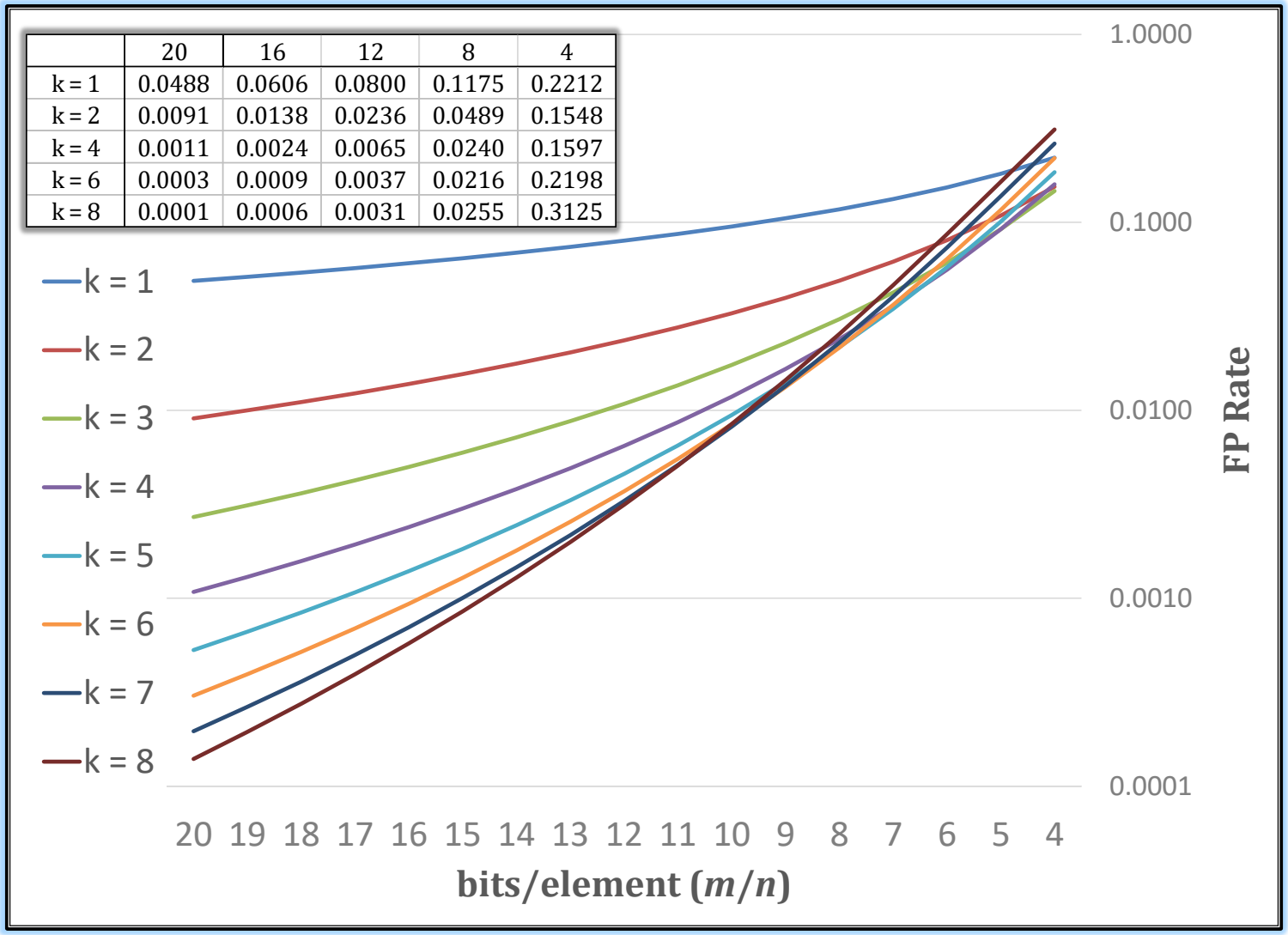
$$\left(1 - \frac{1}{m}\right)^{kn} \approx \mathrm{e}^{-kn/m}$$

- Pr(*false positive*) =

$$\left(1 - \left(1 - \frac{1}{m}\right)^{kn}\right)^{k} \approx \left(1 - \mathrm{e}^{-kn/m}\right)^{k}$$

FALSE POSITIVES VS. (m/n, k)

	20	16	12	8	4
k = 1	0.0488	0.0606	0.0800	0.1175	0.2212
k = 2	0.0091	0.0138	0.0236	0.0489	0.1548
k = 4	0.0011	0.0024	0.0065	0.0240	0.1597
k = 6	0.0003	0.0009	0.0037	0.0216	0.2198
k = 8	0.0001	0.0006	0.0031	0.0255	0.3125



Using sub-hashes as hash functions

- For a good (collision-resistant) hash function:

  larger hash values can be treated as several independent hashes.

  - e.g., SHA-1

  1 x 160 bits, k =1, max(m) = 2^{160}

  5 x 32 bits, k = 5, max(m) = 2^{32}

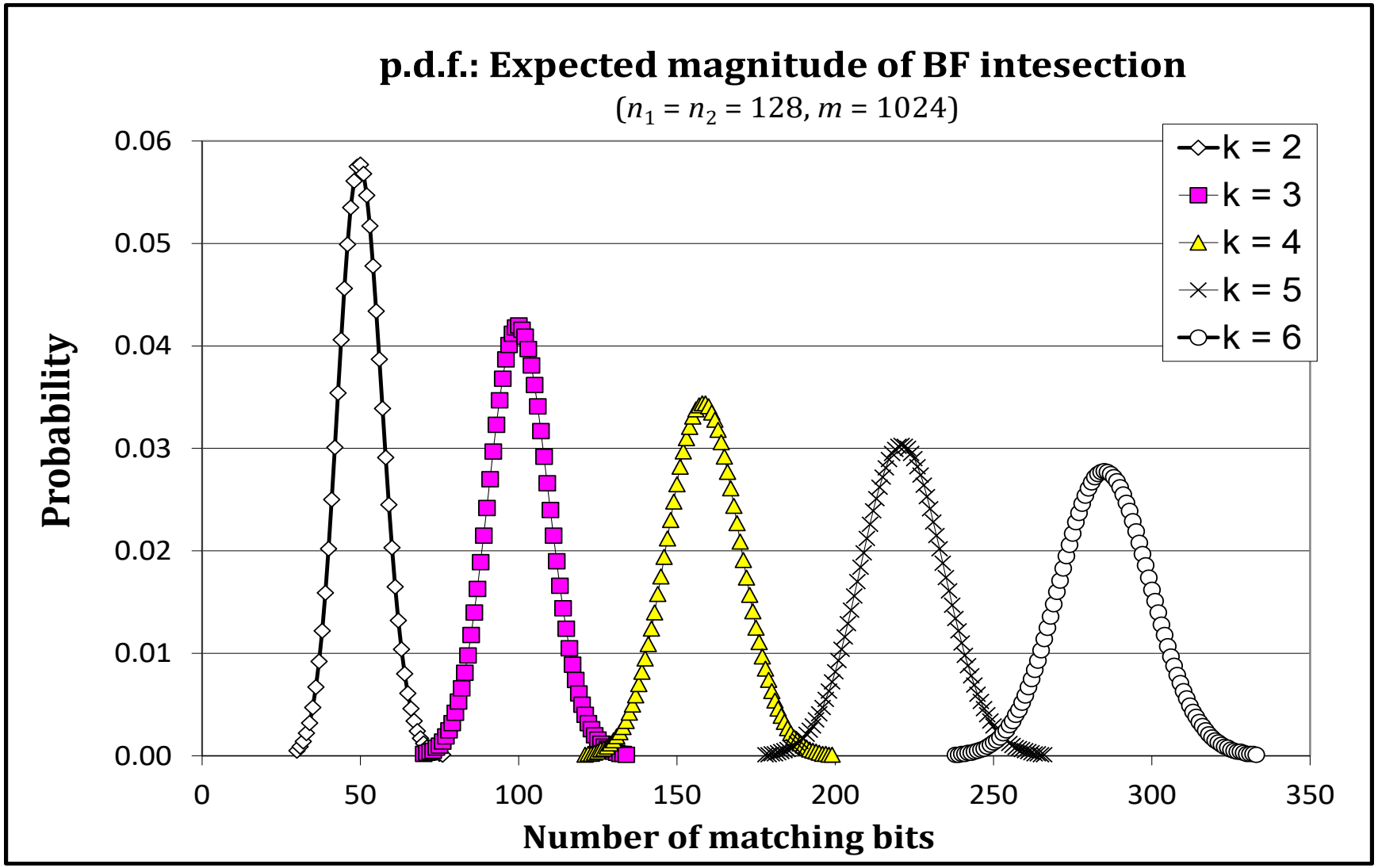
  10 x 16 bits, k = 10, max(m) = 2^{16}

Standard Bloom filter tricks

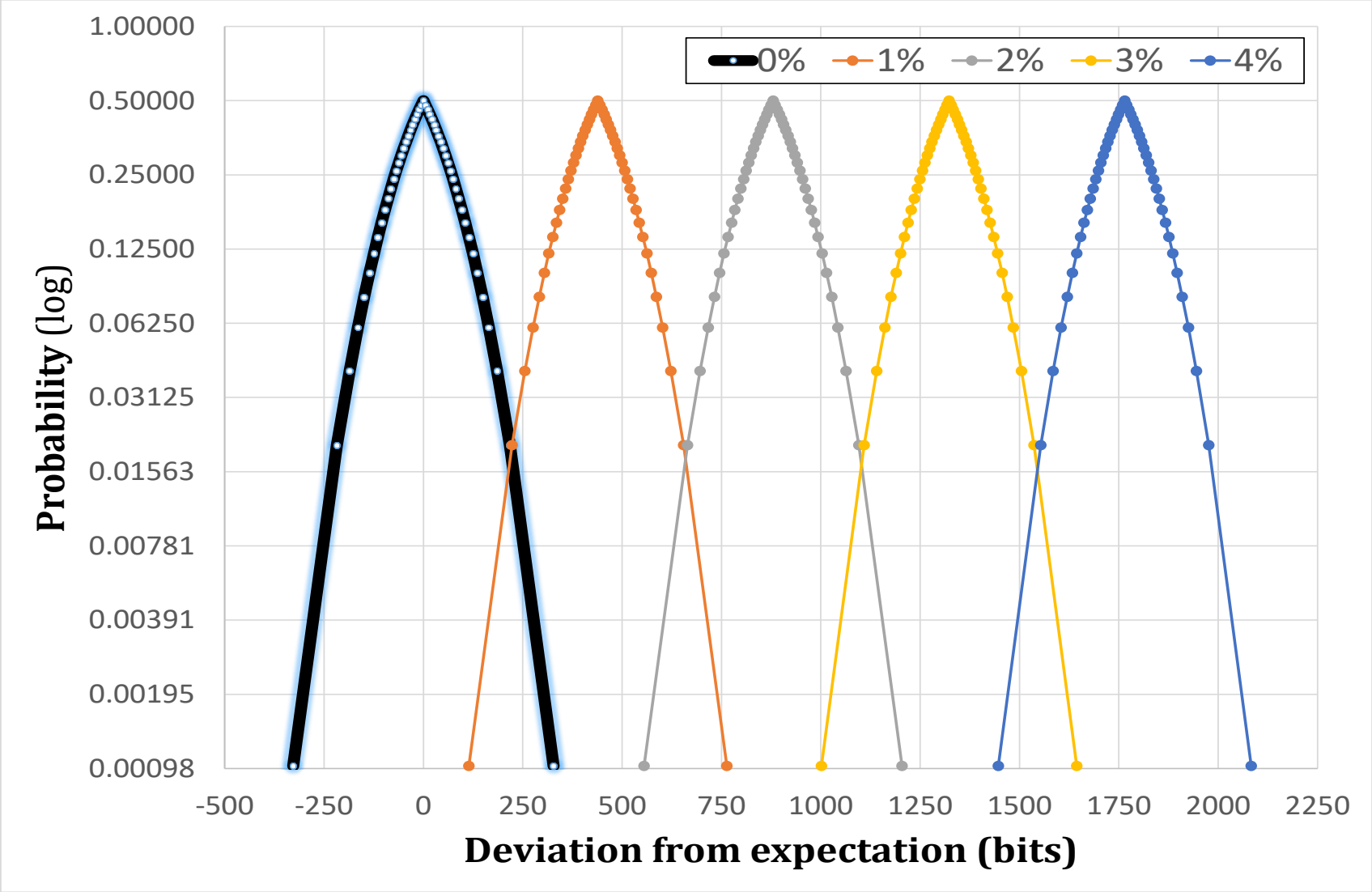
- Expected number of zero bits
  - $m(1 - 1/m)^{kn}$
- Union ➔ bitwise OR
  - same hashes, $m_1 = m_2$, $n = n_1 + n_2$
- Folding
  - union of BF sections; e.g, fold in half:
    - $S_{new}[i] = S_{old}[i] \mid S_{old}[2*i]$, $i = 1..m/2$
    - $m_{new} = m/2$, $h_{new}(x) = h_{old}(x)/2$
- Intersection estimate

$$\frac{1}{m}\left(1-\frac{1}{m}\right)^{-k|S_1 \cap S_2|} \approx \frac{Z_1 + Z_2 - Z_{12}}{Z_1 Z_2}$$

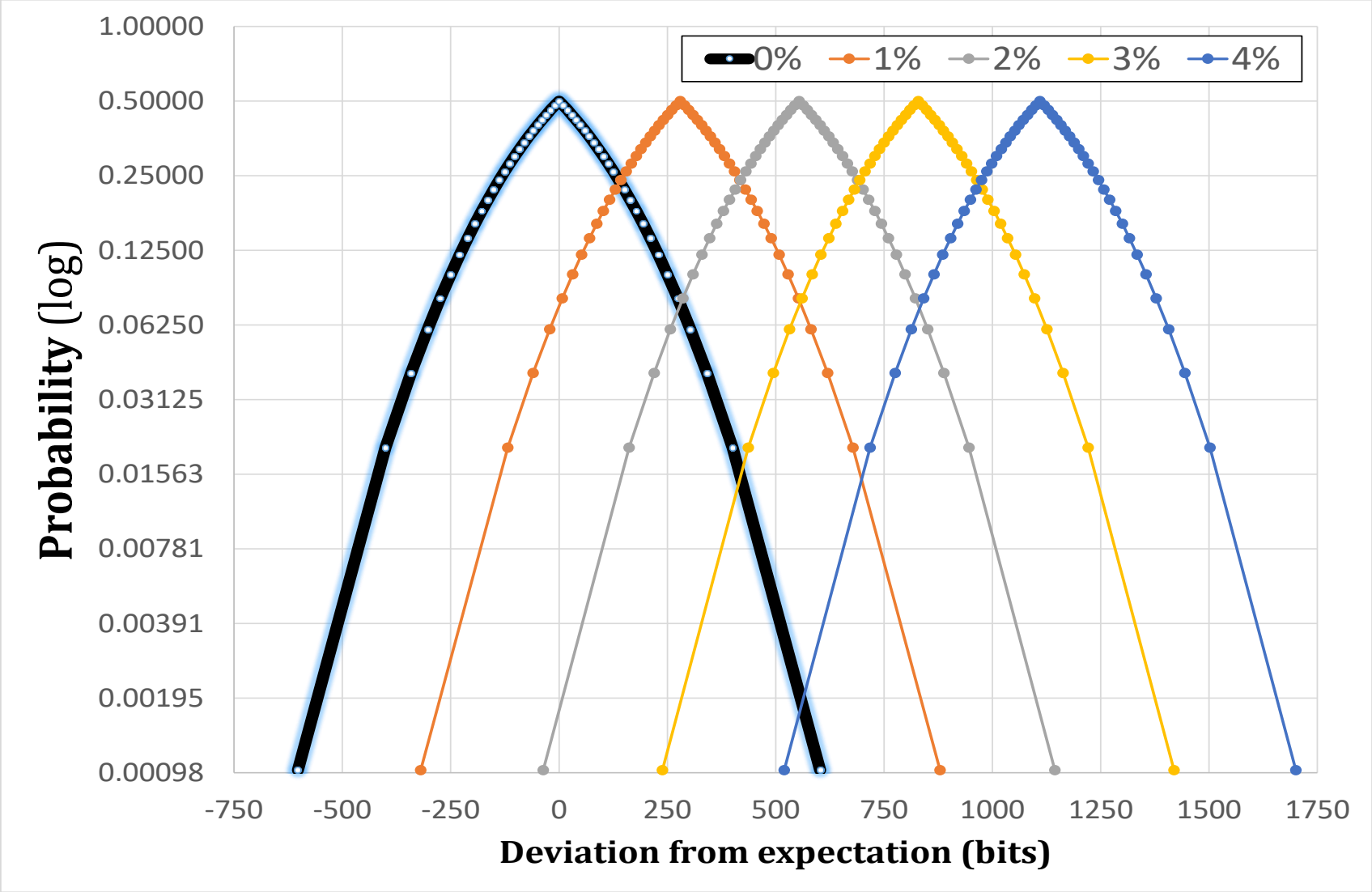
EMPIRICAL COMPARISON OF RANDOM BFs



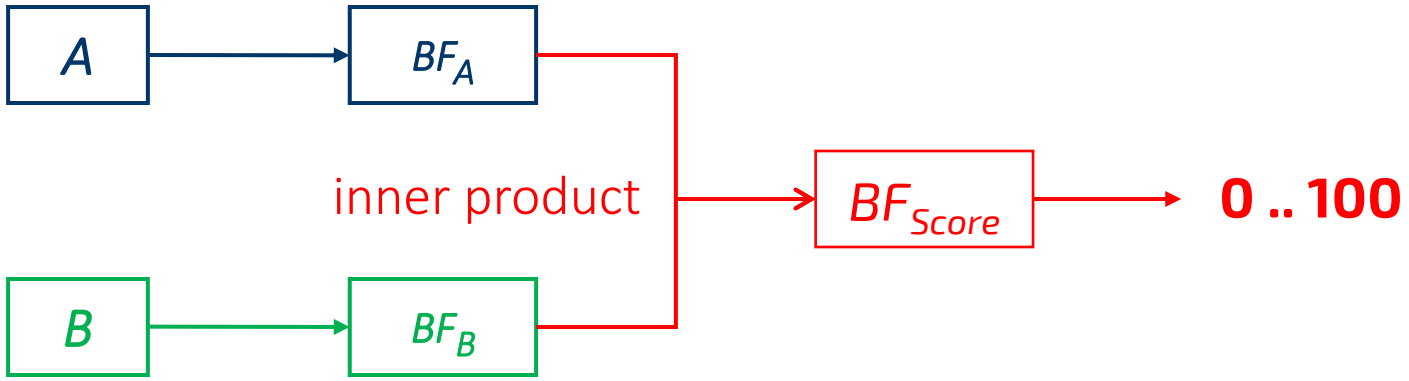
CONTROLLED OVERLAP: m/n = **16**



CONTROLLED OVERLAP: m/n = **4**



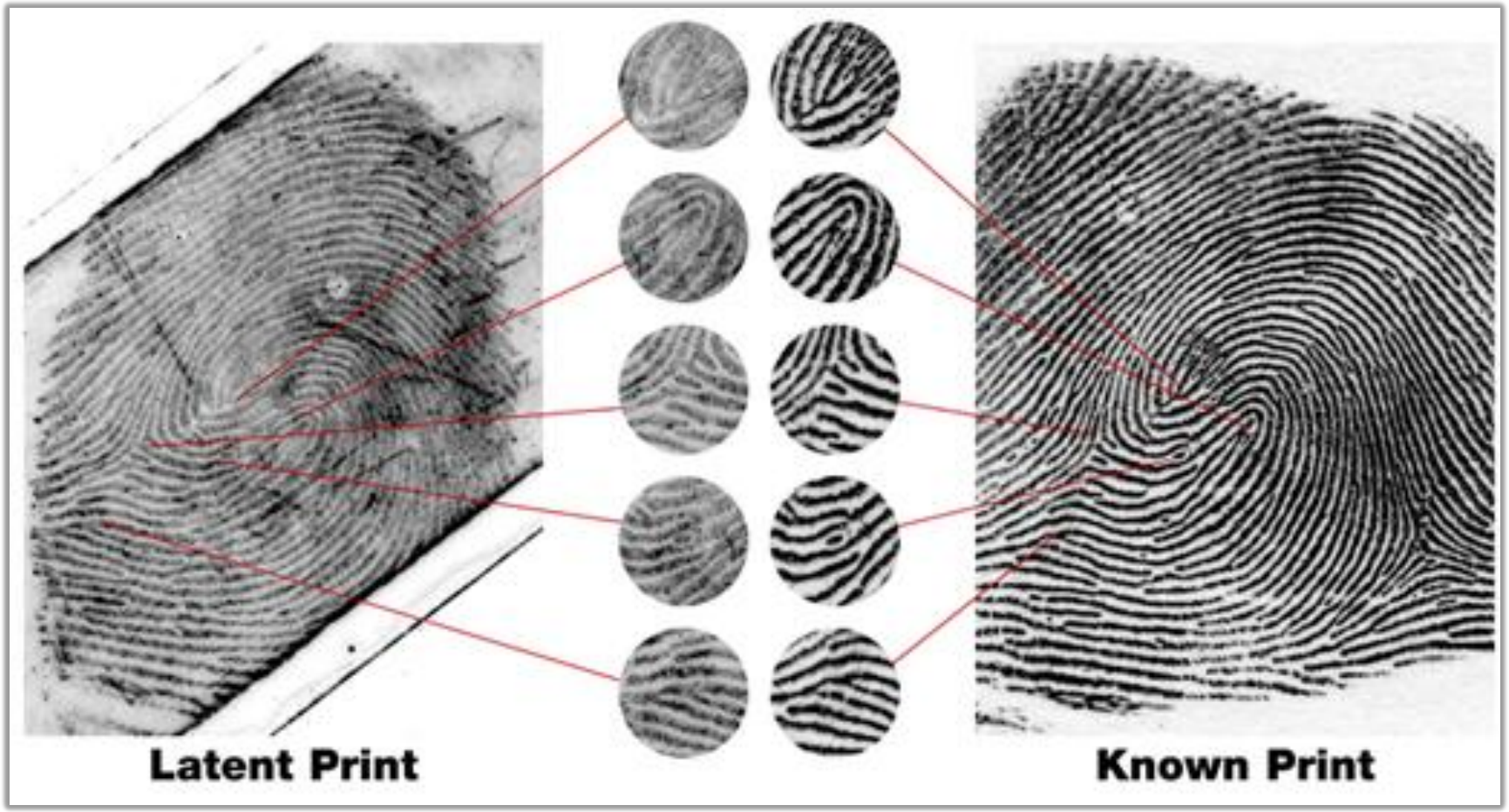
Bloom filter comparison



- ◊ BF overlap due to chance is analytically predictable.
  - » Additional BF overlap is proportional to overlap in features.

➔ BF_{Score} is tuned such that $BF_{Score}(A_{random}, B_{random}) = 0$.

FINGERPRINTING: THE ORIGINAL

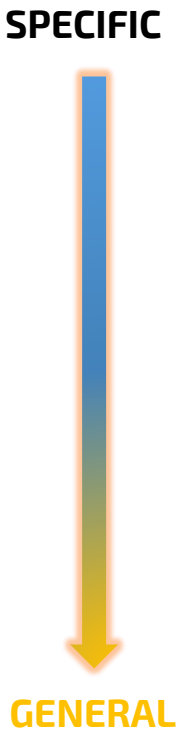


FINGERPRINTING: **THE IDEA**

- Generation
  - pick reference points
  - extract characteristic features
  - label & store for reference
- Comparison
  - extract features from latent
  - calibrate & compare w/ reference set
  - count commonality & interpret

DATA FINGERPRINTING

- How do we apply this to data objects?
  - i.e., how do we pick features?
- a. Interpret the data (most specific)
  - perceptual hashing (a la shazam)
- b. Parse the data
  - split into pieces using known syntax (pdf, zip, html)
    - no interpretation
- c. Treat it as bytes
  - every bit counts → md5, sha-1/2/3
  - find common substrings (string similarity)

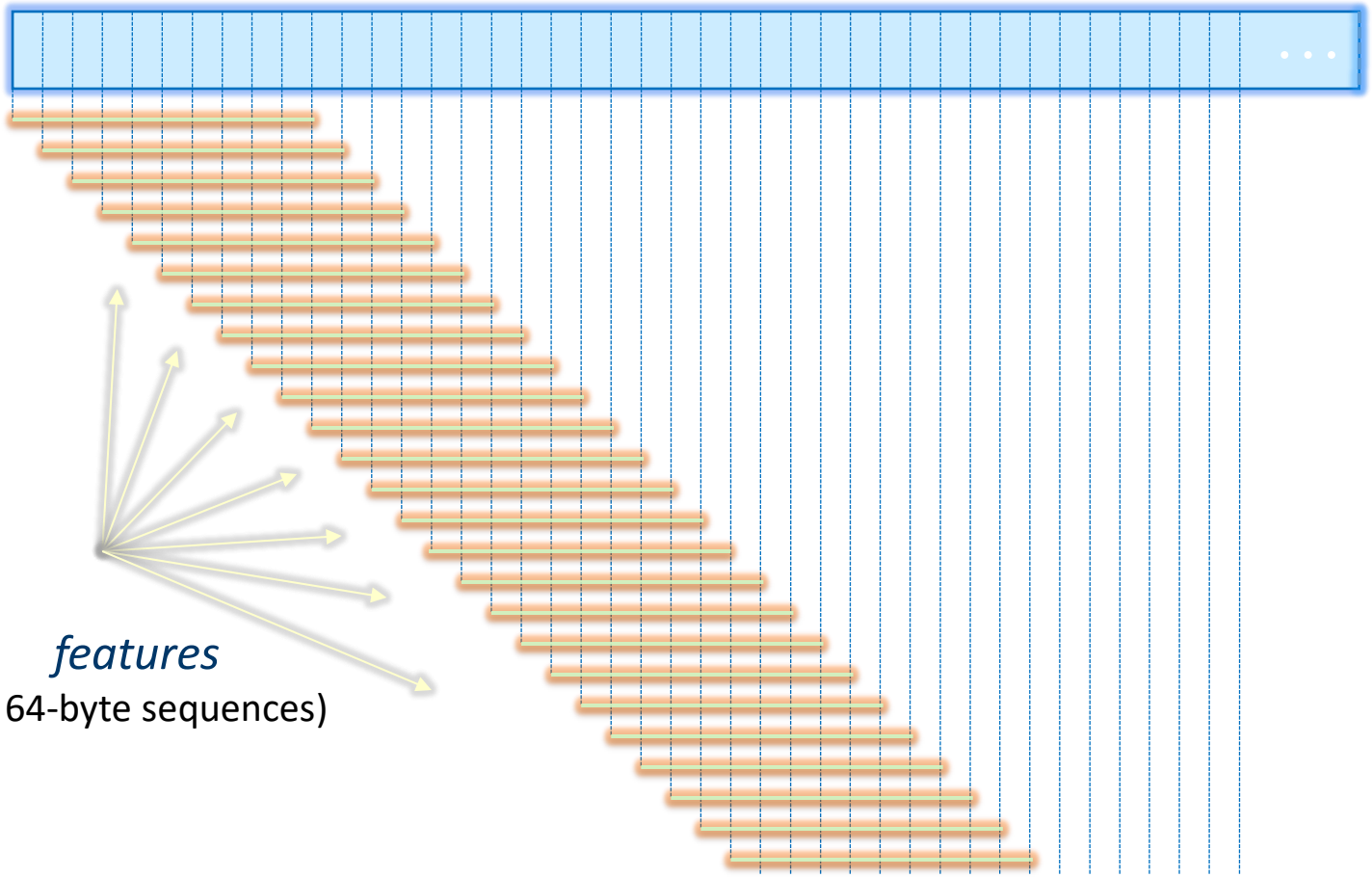


sdhash

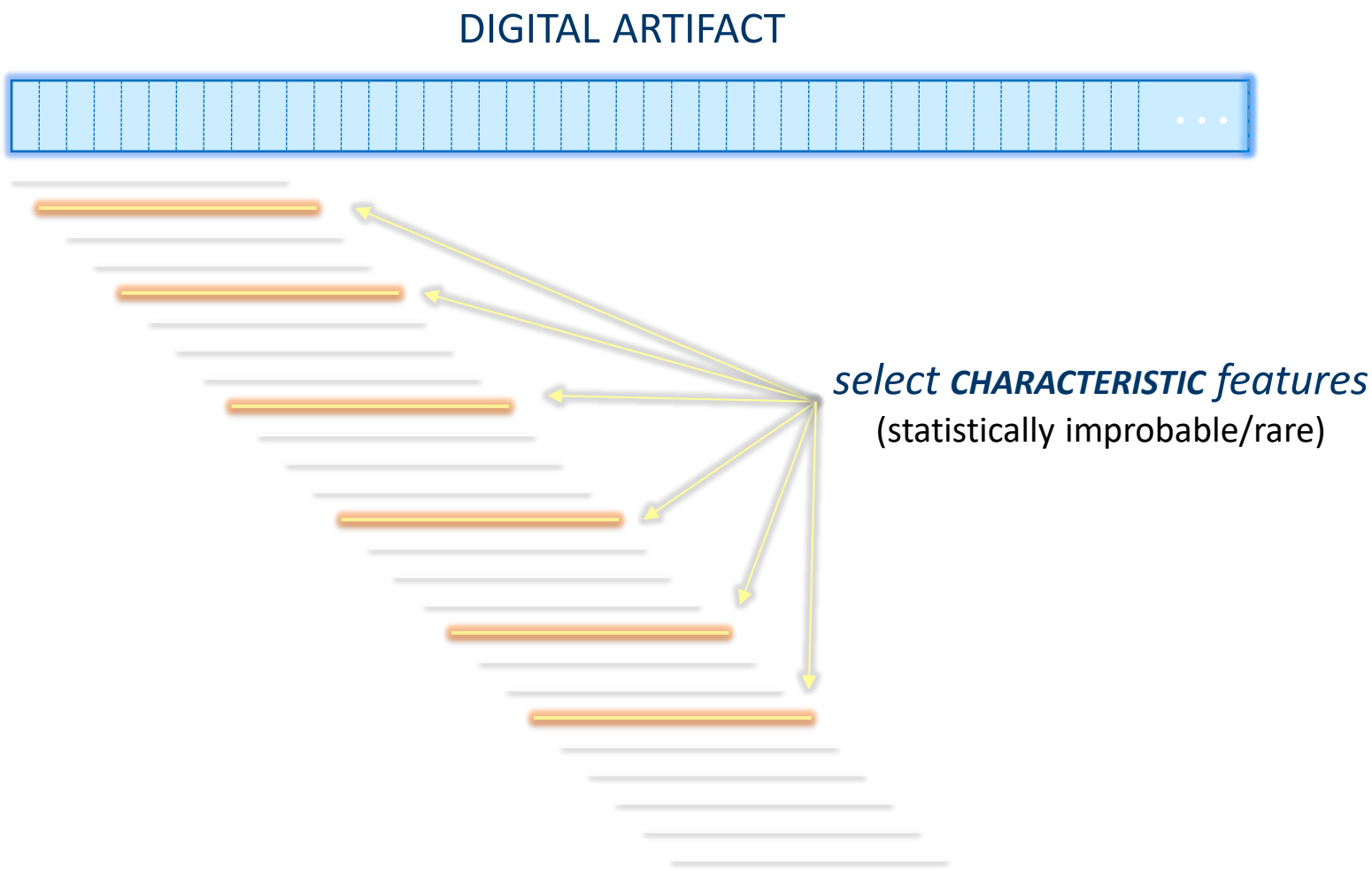
fingerprinting w/ similarity digests

sdhash: FEATURES

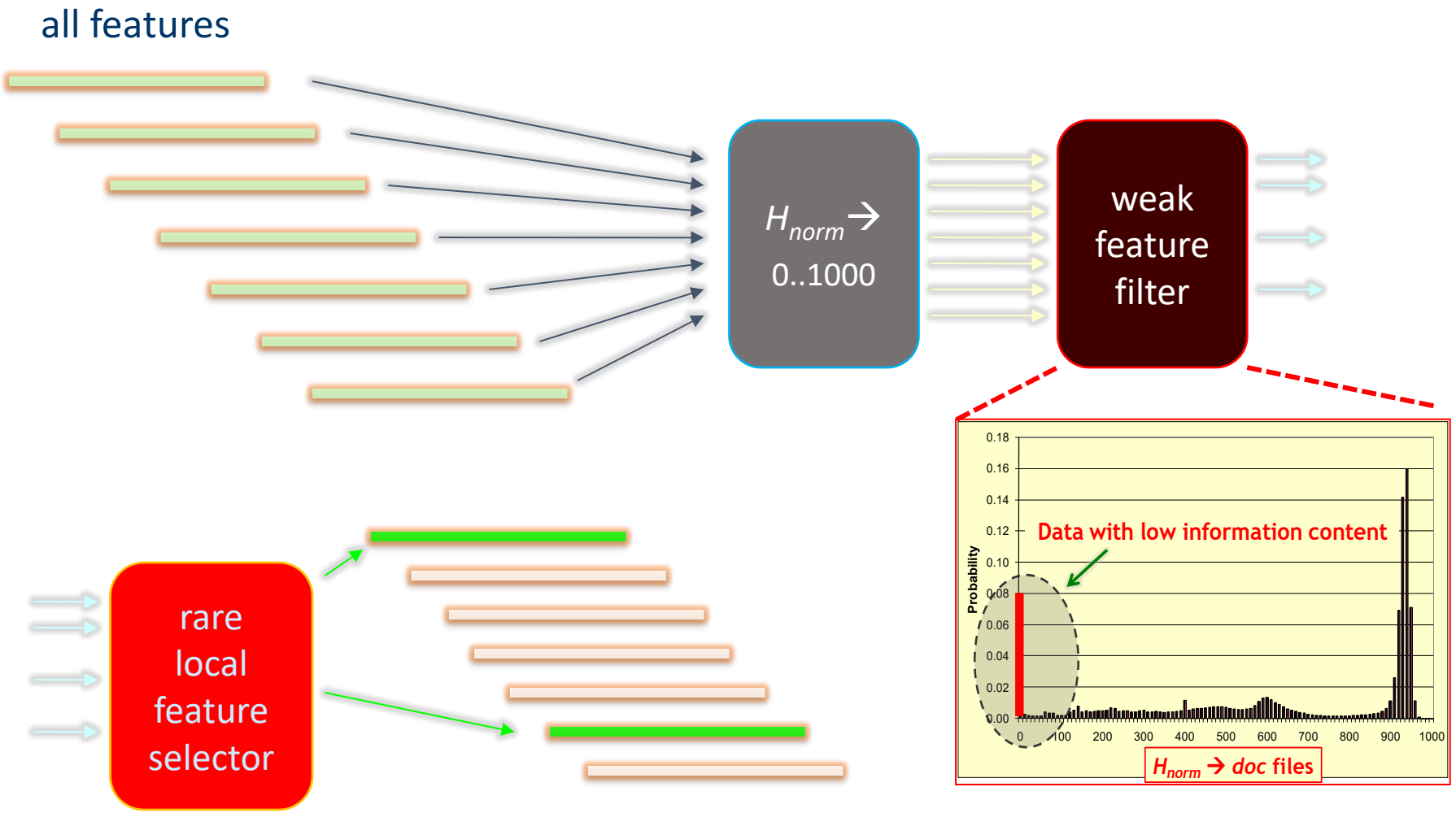
DIGITAL ARTIFACT: block/file/packet/volume as byte stream



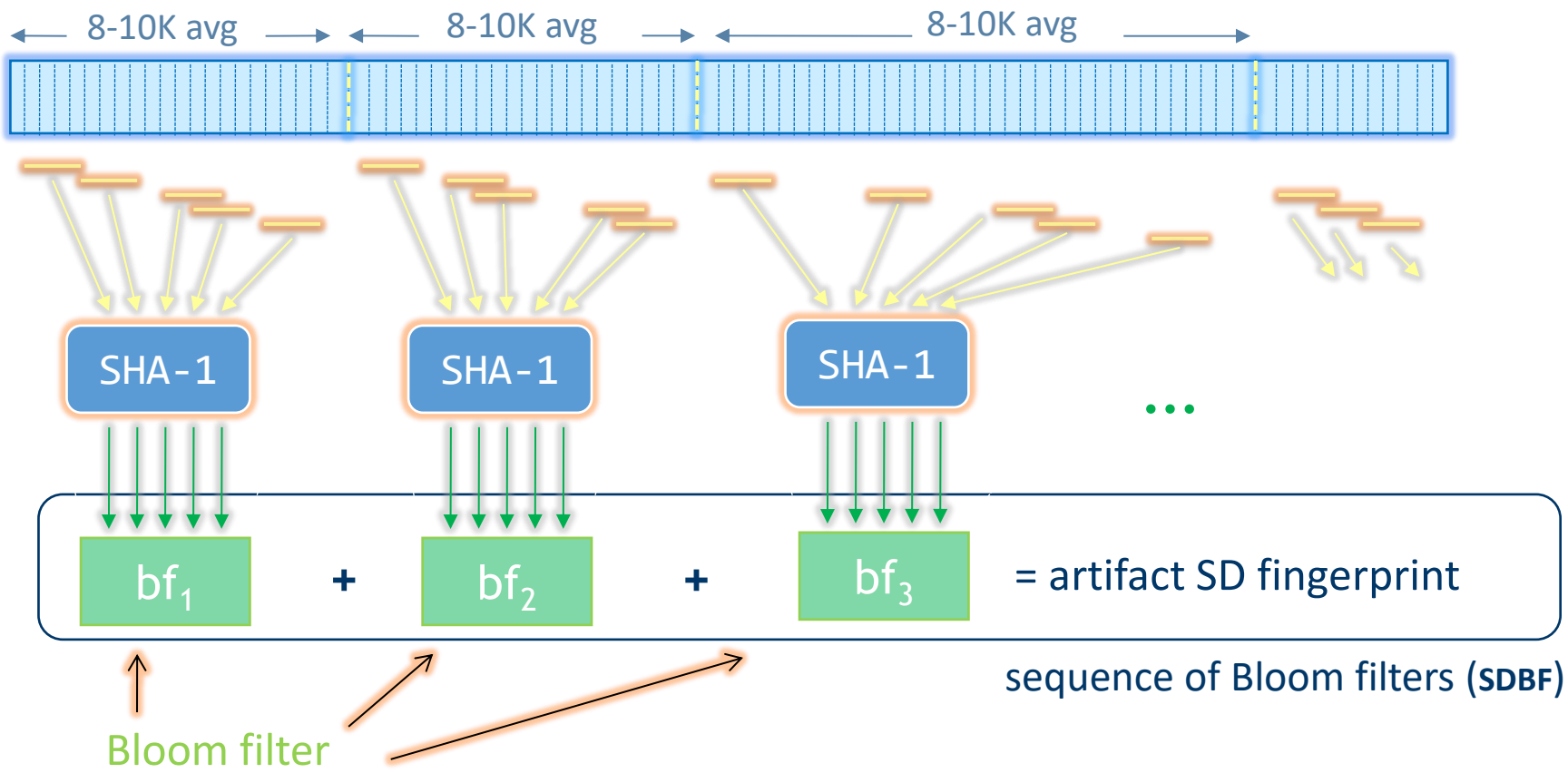
sdhash: CHARACTERISTIC FEATURES



sdhash: FEATURE SELECTION

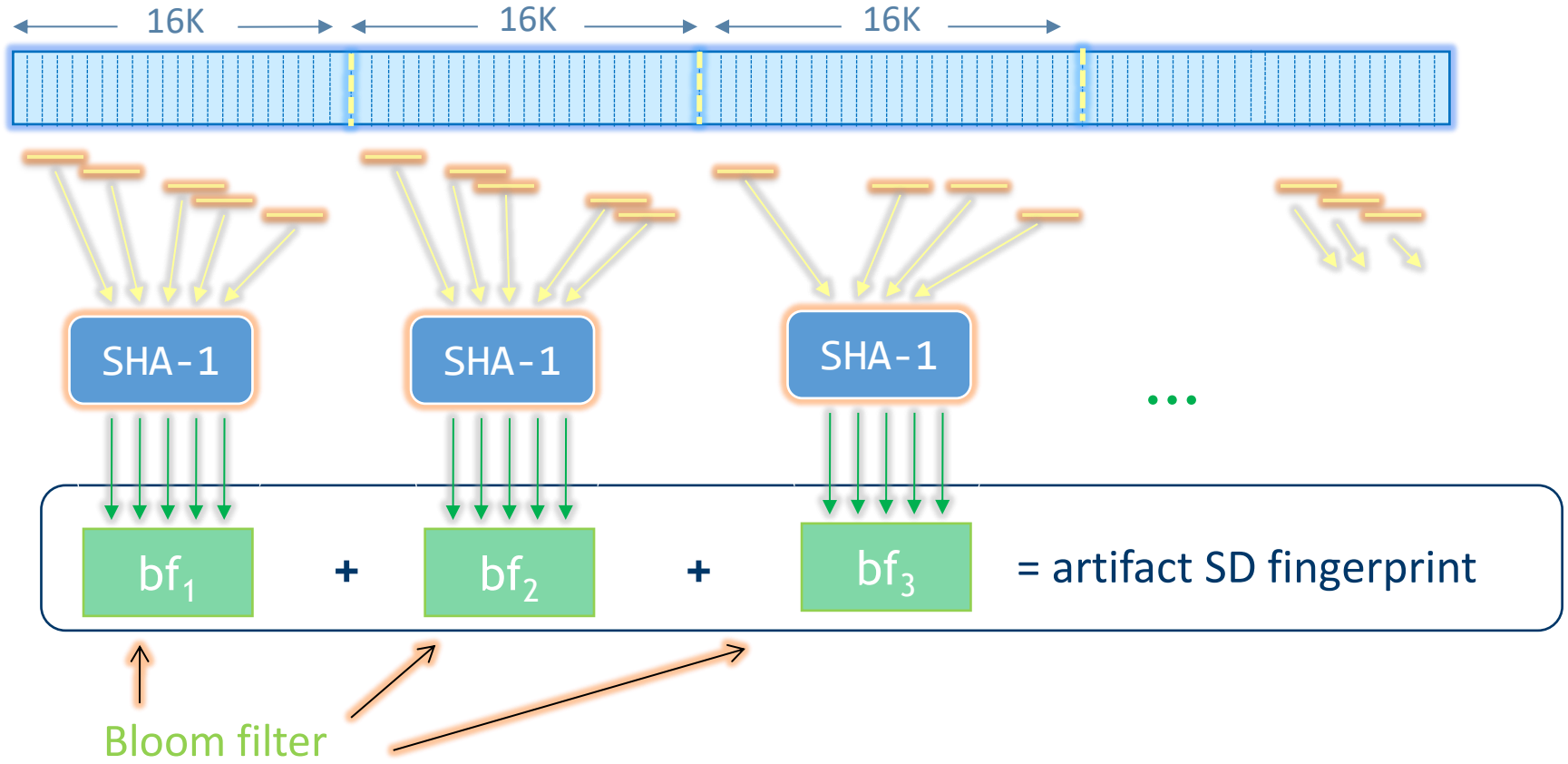


sdhash: SEQUENTIAL FEATURE HASHING & PACKING



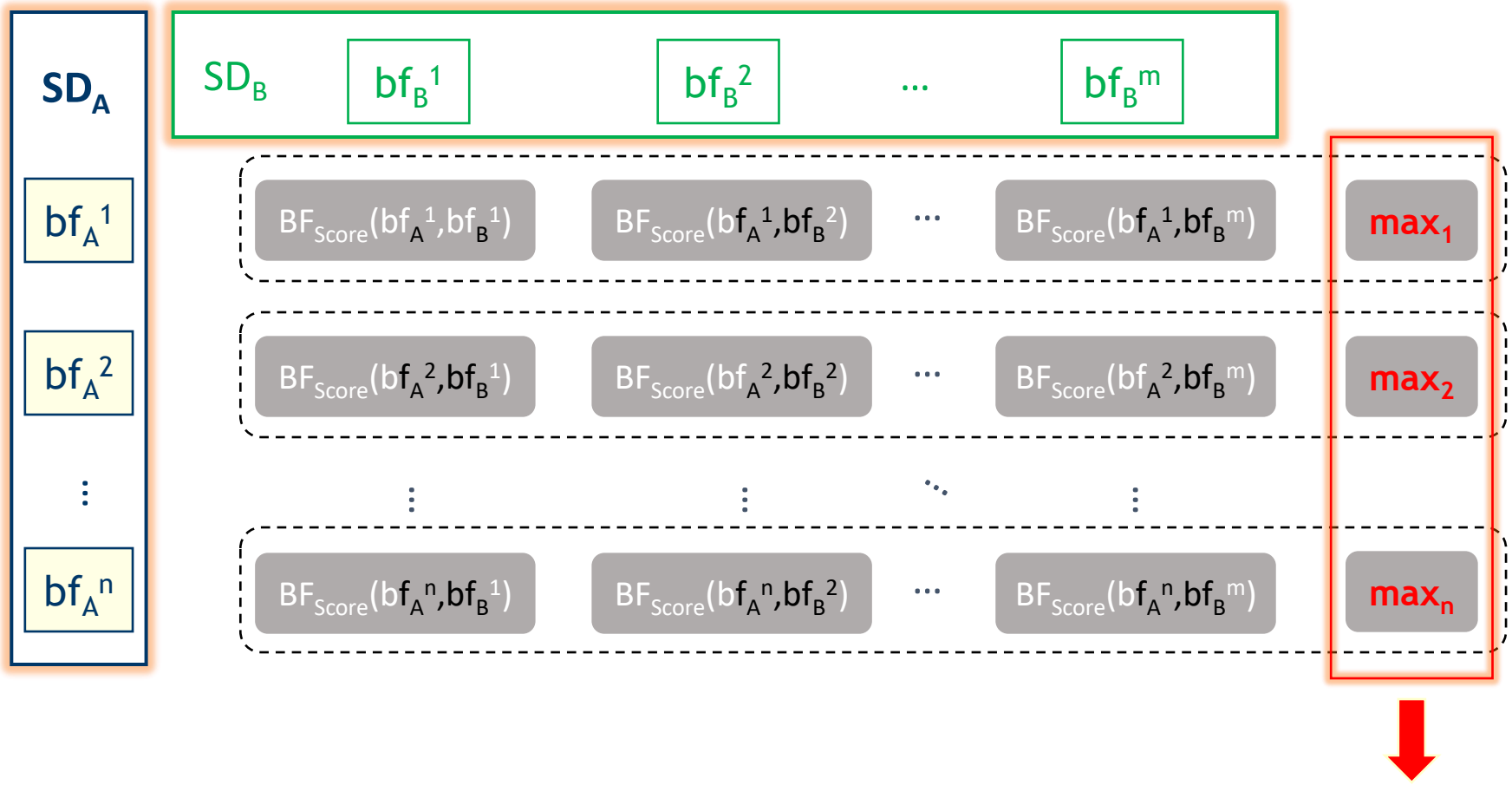
> local SD fingerprint
> 256 bytes
> up to 160 features

sdhash: BLOCK-ALIGNED DIGESTS (PARALLELIZABLE)



> local SD fingerprint
> 256 bytes
> up to 192 features

SIMILARITY DIGEST (SD) COMPARISON



$$SD_{Score}(A,B) = average(max_1, max_2, \ldots, max_n)$$

USEFUL SIMILARITY SCENARIOS

- Resemblance
  - file vs. file
    - file versions/shared embeddings
- Containment
  - file vs. HDD/RAM/network dump
    - simple search for presence
  - block vs. file/dump
    - search for trace data